

Guía Instruccional Clase 2: Programación Orientada a Objetos con Dashboard de Usuarios

Asignatura: Programación II
Institución: Universidad Politécnica Territorial (UPT)
Tema: OOP, CRUD de Usuarios, Bootstrap y Dashboard Profesional
Duración: 3 - 4 Horas Académicas
Prerrequisitos: Clase 1 (Estructura MVC, Composer, Git) completada

1. Objetivos de la Clase

Al finalizar esta sesión, el estudiante será capaz de:

- 1. Comprender los fundamentos de **Programación Orientada a Objetos (POO)** aplicados a PHP.
- 2. Implementar operaciones **CRUD** (Crear, Leer, Actualizar, Eliminar) para gestión de usuarios.
- 3. Diseñar una interfaz moderna y responsiva utilizando **Bootstrap 5**.
- 4. Mantener el flujo de trabajo profesional con **Git/GitHub**.
- 5. Preparar la arquitectura para futuros módulos de autenticación (Login).

2. Introducción Teórica: POO para Novatos (20 Minutos)

Para el Profesor: Use analogías cotidianas. Los estudiantes vienen de programación estructurada y esto puede ser confuso al inicio.

Concepto POO	Analogía del Mundo Real	Ejemplo en Nuestro Sistema
Clase	El plano de una casa	<code>User.php</code> (define qué datos tiene un usuario)
Objeto	La casa construida	<code>\$usuario = new User()</code> (un usuario específico)
Propiedad	Las características (color, puertas)	<code>\$nombre, \$email, \$rol</code>
Método	Las acciones (abrir puerta, pintar)	<code>guardar(), eliminar(), listar()</code>
Interfaz	El contrato de funcionamiento	<code>UserInterface.php</code> (reglas que debe cumplir)

Principios que aplicaremos hoy:

- 1. **Encapsulamiento:** Los datos se protegen dentro de la clase.
- 2. **Responsabilidad Única:** Cada clase hace una cosa bien hecha.
- 3. **Independencia:** El Controlador no sabe cómo se guardan los datos, solo pide que se guarden.

3. Desarrollo Práctico Paso a Paso

FASE 1: Actualización del Entorno Git

Siempre iniciamos sincronizando con el repositorio.

```
git pull origin main
git checkout -b feature/dashboard-usuarios
```

Explicación: Creamos una rama nueva para esta funcionalidad. Es una buena práctica profesional no trabajar directamente sobre **main**.

FASE 2: Instalación de Bootstrap 5

No descargaremos archivos. Usaremos el CDN para mantener el proyecto ligero.

1. Crear layout base (**src/Views/layouts/main.php**):

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dashboard UPT</title>
  <!-- Bootstrap 5 CSS -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <!-- Iconos -->
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-
icons@1.10.0/font/bootstrap-icons.css">
</head>
<body>
  <!-- Barra de Navegación -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-primary mb-4">
    <div class="container">
      <a class="navbar-brand" href="index.php">
        <i class="bi bi-speedometer2"></i> Sistema UPT
      </a>
      <div class="navbar-nav ms-auto">
        <span class="navbar-text text-white">
          <i class="bi bi-person-circle"></i> Admin
        </span>
      </div>
    </div>
  </nav>

  <!-- Contenido Principal -->
  <div class="container">
```

```

        <?php echo $contenido; ?>
    </div>

    <!-- Footer -->
    <footer class="mt-5 py-3 bg-light">
        <div class="container text-center">
            <p class="text-muted mb-0">Programación II - UPT <?php echo
date('Y'); ?></p>
        </div>
    </footer>

    <!-- Bootstrap JS -->
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.
js"></script>
</body>
</html>

```

FASE 3: La Interfaz (Contrato de Código)

Para el Profesor: Explique que la interfaz es como un contrato. Cualquier clase que diga "implemento UserInterface" PROMETE tener estos métodos. Esto permitirá cambiar la base de datos después sin romper el sistema.

1. Crear `src/Interfaces/UserInterface.php`:

```

<?php
namespace App\Interfaces;

interface UserInterface {
    public function listar(): array;
    public function obtenerPorId(int $id): ?array;
    public function crear(array $datos): bool;
    public function actualizar(int $id, array $datos): bool;
    public function eliminar(int $id): bool;
}

```

FASE 4: El Modelo de Usuario (Clase)

Nota: Por ahora usaremos un array en memoria. En la Clase 4 conectaremos la Base de Datos real. Esto permite que los estudiantes prueben sin configurar MySQL aún.

2. Crear `src/Models/User.php`:

```

<?php
namespace App\Models;

use App\Interfaces\UserInterface;

class User implements UserInterface {

    // Simulamos una base de datos en memoria
    private static array $usuarios = [
        ['id' => 1, 'nombre' => 'Admin', 'email' => 'admin@upt.edu.ve', 'rol'
=> 'Administrador'],
        ['id' => 2, 'nombre' => 'Juan Pérez', 'email' => 'juan@upt.edu.ve',
'rol' => 'Estudiante'],
        ['id' => 3, 'nombre' => 'María García', 'email' => 'maria@upt.edu.ve',
'rol' => 'Profesor'],
    ];

    private static int $nextId = 4;

    public function listar(): array {
        return self::$usuarios;
    }

    public function obtenerPorId(int $id): ?array {
        foreach (self::$usuarios as $usuario) {
            if ($usuario['id'] === $id) {
                return $usuario;
            }
        }
        return null;
    }

    public function crear(array $datos): bool {
        $datos['id'] = self::$nextId++;
        self::$usuarios[] = $datos;
        return true;
    }

    public function actualizar(int $id, array $datos): bool {
        foreach (self::$usuarios as &$usuario) {
            if ($usuario['id'] === $id) {
                $usuario = array_merge($usuario, $datos);
                return true;
            }
        }
        return false;
    }

    public function eliminar(int $id): bool {

```

```

        foreach (self::$usuarios as $key => $usuario) {
            if ($usuario['id'] === $id) {
                unset(self::$usuarios[$key]);
                return true;
            }
        }
        return false;
    }
}

```

FASE 5: El Controlador de Usuarios

3. Crear `src/Controllers/UserController.php`:

```

<?php
namespace App\Controllers;

use App\Models\User;

class UserController {

    private User $modelo;

    public function __construct() {
        $this->modelo = new User();
    }

    public function index() {
        $usuarios = $this->modelo->listar();
        require __DIR__ . '/../Views/users/index.php';
    }

    public function crear() {
        require __DIR__ . '/../Views/users/form.php';
    }

    public function guardar() {
        $datos = [
            'nombre' => $_POST['nombre'] ?? '',
            'email' => $_POST['email'] ?? '',
            'rol' => $_POST['rol'] ?? 'Estudiante'
        ];

        $this->modelo->crear($datos);
        header('Location: index.php?controlador=User&action=index');
        exit;
    }
}

```

```

    public function editar($id) {
        $usuario = $this->modelo->obtenerPorId((int)$id);
        require __DIR__ . '/../Views/users/form.php';
    }

    public function eliminar($id) {
        $this->modelo->eliminar((int)$id);
        header('Location: index.php?controlador=User&action=index');
        exit;
    }
}

```

FASE 6: Las Vistas del Dashboard

4. Crear `src/Views/users/index.php` (Lista de Usuarios):

```

<?php ob_start(); ?>

<div class="d-flex justify-content-between align-items-center mb-4">
    <h2><i class="bi bi-people"></i> Gestión de Usuarios</h2>
    <a href="index.php?controlador=User&action=crear" class="btn btn-success">
        <i class="bi bi-plus-lg"></i> Nuevo Usuario
    </a>
</div>

<div class="card shadow-sm">
    <div class="card-body">
        <div class="table-responsive">
            <table class="table table-hover">
                <thead class="table-light">
                    <tr>
                        <th>ID</th>
                        <th>Nombre</th>
                        <th>Email</th>
                        <th>Rol</th>
                        <th>Acciones</th>
                    </tr>
                </thead>
                <tbody>
                    <?php foreach ($usuarios as $user): ?>
                        <tr>
                            <td><?php echo $user['id']; ?></td>
                            <td><?php echo htmlspecialchars($user['nombre']); ?>
                            <td><?php echo htmlspecialchars($user['email']); ?>
                            <td>
                                <span class="badge bg-<?php echo $user['rol'] ===

```



```

        <input type="hidden" name="id" value="<?php echo
$usuario['id']; ?>">
        <?php endif; ?>

        <div class="mb-3">
            <label class="form-label">Nombre Completo</label>
            <input type="text" name="nombre" class="form-control"
                value="<?php echo $usuario['nombre'] ?? ''; ?>"
required>
        </div>

        <div class="mb-3">
            <label class="form-label">Correo Institucional</label>
            <input type="email" name="email" class="form-control"
                value="<?php echo $usuario['email'] ?? ''; ?>"
required>
        </div>

        <div class="mb-3">
            <label class="form-label">Rol</label>
            <select name="rol" class="form-select">
                <option value="Estudiante" <?php echo
($usuario['rol'] ?? '') === 'Estudiante' ? 'selected' : ''; ?
>>Estudiante</option>
                <option value="Profesor" <?php echo
($usuario['rol'] ?? '') === 'Profesor' ? 'selected' : ''; ?>>Profesor</option>
                <option value="Administrador" <?php echo
($usuario['rol'] ?? '') === 'Administrador' ? 'selected' : ''; ?
>>Administrador</option>
            </select>
        </div>

        <div class="d-grid gap-2">
            <button type="submit" class="btn btn-primary">
                <i class="bi bi-save"></i> Guardar
            </button>
            <a href="index.php?controlador=User&action=index"
class="btn btn-secondary">
                Cancelar
            </a>
        </div>
    </form>
</div>
</div>
</div>
</div>
</div>

<?php
$contenido = ob_get_clean();

```



```
require __DIR__ . '/../layouts/main.php';
?>
```

FASE 7: Actualizar el Front Controller (Enrutador Simple)

6. Modificar `public/index.php`:

```
<?php
require_once __DIR__ . '/../vendor/autoload.php';

use App\Controllers\HomeController;
use App\Controllers\UserController;

// Enrutamiento básico
$controlador = $_GET['controlador'] ?? 'Home';
$accion = $_GET['action'] ?? 'index';
$id = $_GET['id'] ?? null;

// Mapeo de controladores
$controladores = [
    'Home' => HomeController::class,
    'User' => UserController::class
];

if (array_key_exists($controlador, $controladores)) {
    $clase = $controladores[$controlador];
    $objeto = new $clase();

    if (method_exists($objeto, $accion)) {
        if ($id) {
            $objeto->$accion($id);
        } else {
            $objeto->$accion();
        }
    } else {
        echo "Acción no encontrada";
    }
} else {
    echo "Controlador no encontrado";
}
```

FASE 8: Commit y Push a GitHub

```
git add .
git commit -m "Feature: Dashboard de usuarios con CRUD completo y Bootstrap"
```

```
git push origin feature/dashboard-usuarios
```

Para fusionar con main (opcional en clase):

```
git checkout main
git merge feature/dashboard-usuarios
git push origin main
```

4. Prueba del Sistema

1. Ejecutar servidor: `php -S localhost:8000 -t public`
2. Navegar a: `http://localhost:8000`
3. Probar flujo completo:
 - Ver lista de usuarios
 - Crear nuevo usuario
 - Editar usuario existente
 - Eliminar usuario

5. Checklist de Evaluación

Criterio	Descripción	Puntos
POO	Clases correctamente estructuradas con namespace	25%
Interfaz	UserInterface implementada en el Modelo	20%
CRUD	Las 4 operaciones funcionan correctamente	25%
Bootstrap	Interfaz responsiva y profesional	15%
Git	Rama feature creada y commits descriptivos	15%

6. Roadmap del Curso (Próximas Clases)

Clase	Tema	Estado
1	Estructura MVC, Composer, Git	☒ Completado
2	POO, CRUD Usuarios, Bootstrap	☒ Esta Clase
3	Conexión a MySQL con PDO	☐ Próximo
4	Sistema de Login y Sesiones	☐ Futuro

Clase	Tema	Estado
5	Middleware y Seguridad	 Futuro
6	Proyecto Final Integrador	 Futuro

7. Notas para el Profesor

1. **Sobre los datos en memoria:** Explique que esto es temporal. Cuando conectemos MySQL, solo cambiaremos la clase **User**, el Controlador y las Vistas seguirán igual. Eso es la magia de las Interfaces.
 2. **Sobre Bootstrap:** No necesitan descargar nada. El CDN es suficiente para aprendizaje. En producción real, podrían compilar sus propios assets.
 3. **Sobre Seguridad:** Advierta que este sistema NO tiene validación de seguridad ni login. Cualquier persona puede eliminar usuarios. Eso se corregirá en la Clase 4 con el sistema de autenticación.
 4. **Errores Comunes:**
 - **Namespace incorrecto:** Verificar que coincide con la estructura de carpetas.
 - **Formulario no envía datos:** Revisar que el **action** del form apunte correctamente al controlador.
 - **Git merge conflicts:** Si trabajan en equipo, enseñar a resolver conflictos básicos.
-

8. Tarea para la Próxima Clase

"Personaliza tu Dashboard"

1. Agregar un campo adicional al usuario (ej: **telefono** o **fecha_registro**).
2. Cambiar los colores del tema Bootstrap (usar clases **bg-dark**, **bg-success**, etc.).
3. Agregar un buscador en la tabla de usuarios.
4. Subir todos los cambios a GitHub con commits adecuados.

Entrega: Enlace al repositorio GitHub con la rama actualizada.

"Estudiantes, hoy han construido algo que se parece mucho a un sistema real. Mantengan este estándar de código, usen Git para todo, y en las próximas clases haremos que esto se conecte a una base de datos real y tenga seguridad profesional."